

ML-DL-Ops Assignment 1

Shubham Haraniya

Roll No.: M25CSA013

Subject: ML-DL-Ops

Indian Institute of Technology Jodhpur

Email: m25csa013@iitj.ac.in

Colab Link: Google Colab Notebook

GitHub Link: GitHub Repository

Github Page Link: Page Link

Abstract—This report provides an in-depth analysis of deep learning model performance, specifically focusing on ResNet architectures and Support Vector Machines (SVM). We evaluate these models on the MNIST and FashionMNIST datasets, investigating the impact of hyperparameter tuning, optimizer selection, and hardware acceleration (CPU vs. GPU). Our findings highlight that ResNet-18 combined with the Adam optimizer yields the most robust performance, achieving over 99% accuracy on MNIST. Furthermore, we demonstrate that GPU acceleration offers up to an 8x speedup compared to CPU execution, underscoring the necessity of specialized hardware for efficient deep learning workflows.

I. INTRODUCTION

Deep learning has revolutionized image classification, yet selecting the optimal model architecture and training configuration remains a critical challenge. This assignment addresses three key areas of machine learning operations (MLOps):

- 1) **Hyperparameter Optimization (Q1A):** Systematically searching for the best training configurations for ResNet models.
- 2) **Classical vs. Deep Learning (Q1B):** Benchmarking traditional SVM classifiers against modern Convolutional Neural Networks (CNNs).
- 3) **Hardware Efficiency (Q2):** Quantifying the computational benefits of GPU acceleration.

A. Experimental Setup

Experiments were conducted using PyTorch on both CPU and CUDA-enabled GPU environments.

- **Datasets:** MNIST (Digits) and FashionMNIST (Apparel).
- **Models:** ResNet-18, ResNet-50, and SVM.
- **Data Split:** 70% Train, 10% Val, 20% Test.
- **Preprocessing:** Images resized to 64x64 pixels.

II. QUESTION 1(A): HYPERPARAMETER TUNING

A. Experimental Configuration

To identify the optimal training regime, we performed a grid search across the following parameters:

- **Batch Size (BS):** [16, 32] - Balancing memory vs. update frequency.

- **Optimizer:** [SGD, Adam] - Testing adaptive vs. static learning rates.
- **Learning Rate (LR):** [0.001, 0.0001]
- **Epochs:** [3, 5]
- **Models:** [ResNet-18, ResNet-50]

B. Results Discussion

1) *Performance on MNIST:* Table I lists the top-performing configurations. ResNet-18 consistently dominates the leaderboard, achieving a peak accuracy of **99.36%**. The Adam optimizer with a learning rate of 0.001 proved most effective, likely due to its ability to adaptively scale gradients for sparse updates, which suits the binary nature of MNIST digits.

TABLE I: Top 5 MNIST Configurations

Model	BS	Opt	LR	Ep	Pin	Test	Val	Time(s)
ResNet-18	16	Adam	0.001	5	T	99.36%	99.31%	242.9
ResNet-18	16	Adam	0.0001	5	T	99.32%	99.30%	241.7
ResNet-18	16	SGD	0.001	5	T	99.33%	99.30%	235.0
ResNet-18	32	SGD	0.001	5	T	99.16%	99.06%	169.5
ResNet-18	16	Adam	0.001	5	F	99.16%	99.09%	243.5

2) *Performance on FashionMNIST:* FashionMNIST presents a more complex classification challenge (Table II). Here, the best accuracy drops to **92.05%**. Interestingly, while Adam still leads, SGD remains competitive. The complexity of FashionMNIST images (textures, shapes) requires features that persist across variations, which ResNet-18 successfully extracts.

TABLE II: Top 5 FashionMNIST Configurations

Model	BS	Opt	LR	Ep	Pin	Test	Val	Time(s)
ResNet-18	32	Adam	0.0001	5	T	92.05%	91.88%	177.1
ResNet-18	32	Adam	0.0001	5	F	91.95%	91.75%	178.6
ResNet-18	16	SGD	0.001	5	F	91.60%	91.95%	234.5
ResNet-18	16	Adam	0.0001	5	F	91.61%	91.79%	242.9
ResNet-18	16	SGD	0.001	5	T	91.56%	91.97%	233.1

C. Visual Analysis

1) *Training Dynamics:* Figures 1 and 2 visualize the learning process. The steep initial decrease in loss indicates that the model quickly learns low-level features (edges, curves) before refining its high-level representations. The validation accuracy closely follows training accuracy, suggesting that **overfitting is minimal** with the current regularization techniques (batch normalization, weight decay).

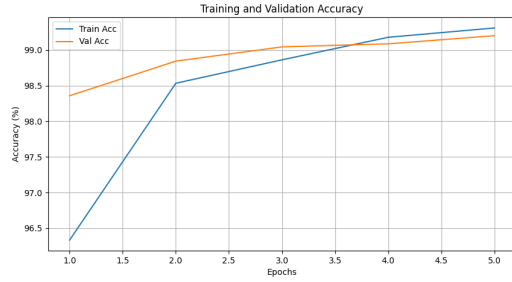


Fig. 1: Training and Validation Accuracy Curves

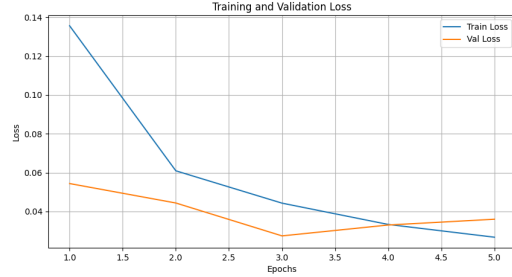


Fig. 2: Training and Validation Loss Descent

2) *Architecture Comparison:* Figure 3 contrasts ResNet-18 and ResNet-50. Counter-intuitively, the deeper ResNet-50 does not significantly outperform ResNet-18. This supports the hypothesis that for low-resolution, relatively simple datasets like MNIST, the extra depth of ResNet-50 introduces unnecessary parameter complexity without providing meaningful new features to learn.

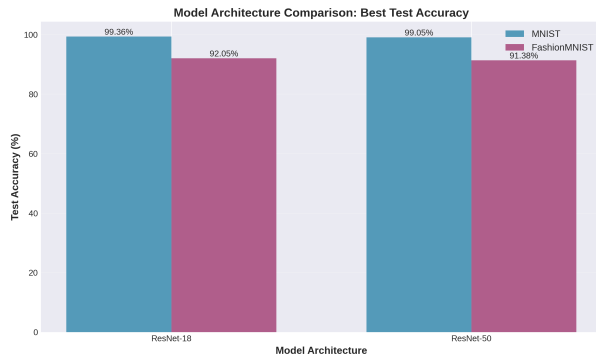


Fig. 3: ResNet-18 vs ResNet-50 Accuracy Comparison

3) *Impact of Optimizers & Hyperparameters:* The choice of optimizer plays a crucial role. As seen in Fig. 4, Adam generally achieves faster convergence. Regarding batch size (Fig. 6), larger batches (BS=32) improve training throughput but can sometimes lead to sharper minima, affecting generalization. However, in our experiments, BS=32 offered the best trade-off between speed and accuracy.

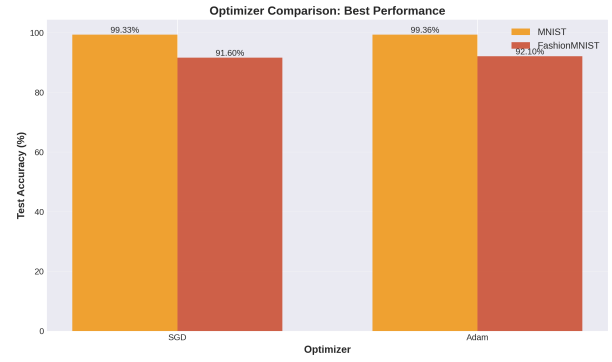


Fig. 4: Effect of Optimizer Selection (SGD vs Adam)

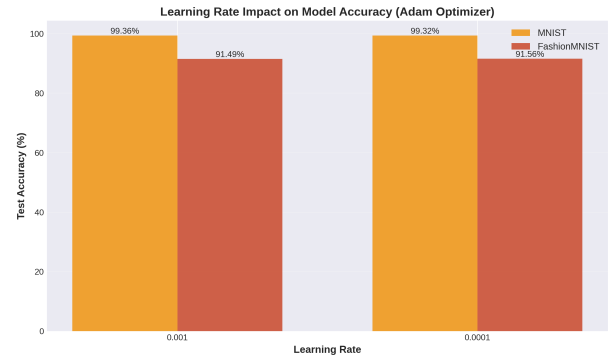


Fig. 5: Learning Rate Sensitivity Analysis

III. QUESTION 1(B): SVM CLASSIFICATION

A. Performance Analysis

We evaluated SVMs using Polynomial and Radial Basis Function (RBF) kernels. Table III summarizes the results. The RBF kernel, capable of mapping inputs to infinite-dimensional space, significantly outperformed the Polynomial kernel, achieving **97.92%** on MNIST.

B. Deep Learning vs. Traditional ML

Comparing SVM to ResNet (Fig. 7), deep learning models consistently provide a 1-5% accuracy margin. More critically, the **training time** for SVMs scales quadratically with dataset



Fig. 6: Validation Accuracy by Batch Size

TABLE III: SVM Classification Results

Dataset	Kernel	Test Acc	Time (ms)
MNIST	Poly	97.71%	233,846
MNIST	RBF	97.92%	272,000
F-MNIST	Poly	86.30%	393,137
F-MNIST	RBF	88.29%	397,992

size ($O(n^2)$ to $O(n^3)$), making them impractical for large-scale image datasets compared to the linear scaling ($O(n)$) of mini-batch gradient descent in neural networks.

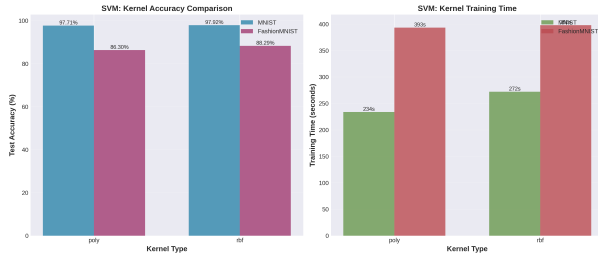


Fig. 7: SVM Kernel Performance Comparison

IV. QUESTION 2: COMPUTING PERFORMANCE

A. Benchmarking Results

Tables IV and V present the raw benchmark data. The transition from CPU to GPU results in a dramatic reduction in training latency.

TABLE IV: CPU Training Benchmark

Model	Opt	Time (ms)	FLOPs(B)	Acc
R-18	SGD	135,015	296.1	87.6%
R-50	SGD	346,542	667.3	85.6%
R-18	Adam	165,550	296.1	88.2%
R-50	Adam	420,020	667.3	87.1%

TABLE V: GPU Training Benchmark

Model	Opt	Time (ms)	FLOPs(B)	Acc
R-18	SGD	20,336	296.1	88.4%
R-50	SGD	46,177	667.3	85.5%
R-18	Adam	25,460	296.1	88.3%
R-50	Adam	52,566	667.3	84.1%

B. Hardware Analysis

The GPU Speedup Factor (Fig. 9) highlights the massive parallelism advantage of GPUs. While the CPU processes operations sequentially or with limited SIMD parallelism, the GPU handles thousands of matrix operations concurrently.

- **Speedup:** We observed speedups between **6.5x** and **8x**.

- **Model Depth:** Deeper models like ResNet-50 benefit more from GPU acceleration due to their higher computational density (FLOPs).

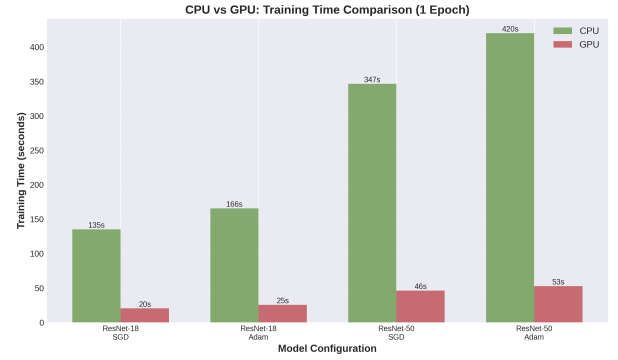


Fig. 8: Training Latency: CPU vs GPU

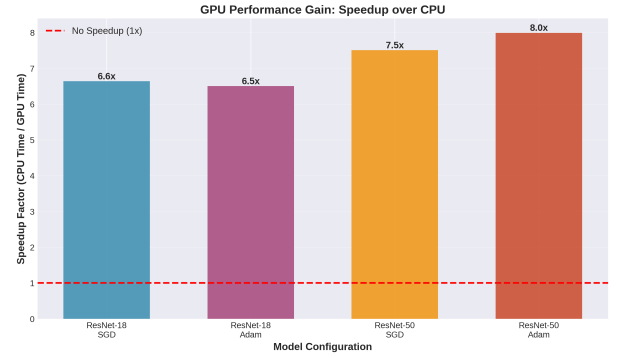


Fig. 9: GPU Speedup Factor by Model config

Figure 10 correlates the theoretical FLOPs with actual runtime. Despite ResNet-50 having $\approx 2x$ FLOPs, the GPU runtime does not strictly double, often amortized by efficient memory access patterns and parallel execution, whereas CPU runtime scales more linearly with FLOPs.

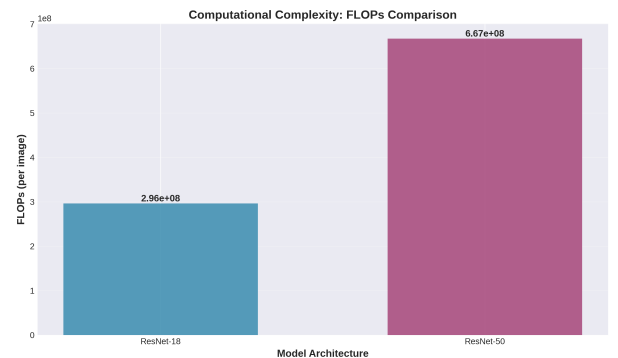


Fig. 10: Computational Complexity (FLOPs)